

PRIVILEGE ESCALATION TRADECRAFT ANALYSIS CHEAT SHEET

Detection Logics

UAC Bypass Detections

Area	What Students Must Understand	How to Reason About It (Detection Mindset)
Overall UAC bypass mindset	UAC bypass = getting high-integrity behavior from a medium-integrity context without a normal UAC prompt.	Ask: "What changed in integrity, parent/child chain, registry, DLLs, or ETW events when the bypass happened?"
Integrity levels & parent/child relationships	Difference between Low / Medium / High / System integrity and how they apply to processes.	Look for Medium parent → High child transitions; check if this is normal for that parent or a sign of elevation abuse.
Stable process identifiers	process_guid vs process_id , and why PIDs can be reused.	Always correlate with GUIDs for parent/child joins to avoid false links due to PID reuse.
Auto-approved executables (AutoApproveEXEList)	Some Windows binaries (e.g. fodhelper.exe , sdclt.exe , computerdefaults.exe) auto-elevate for admins.	Treat unexpected children of these EXEs (e.g., cmd.exe , powershell.exe , odd tools) as strong UAC bypass indicators.
Auto-approved EXE parent behavior	Auto-approved EXEs are "normal" elevation paths but can be hijacked.	Ask: "Is this child something this tool normally spawns, or does it look like arbitrary code execution via UAC bypass?"
Elevated COM classes & dllhost.exe	Some COM classes run elevated and are hosted by dllhost.exe /ProcessId:{CLSID} .	Monitor dllhost.exe command lines for specific CLSIDs tied to elevated COM; unusual activations can signal bypass attempts.
CLSID awareness	CLSIDs for known elevated COM objects (e.g. ICMLuaUtil) are valuable IoCs.	Map CLSIDs ↔ COM objects; question why a particular elevated CLSID is being activated and by which process.
Registry hijack: ms-settings / FodHelper	UAC bypass via HKCU\Software\Classes\ms-settings\Shell\Open\command and DelegateExecute .	Any write to this key by a user process is suspicious; check if default command points to unusual binaries (cmd, calc, custom).

Area	What Students Must Understand	How to Reason About It (Detection Mindset)
Importance of HKCU in UAC bypass	Per-user registry (HKCU) can control behavior of elevated components.	Don't ignore HKCU changes; they can redirect auto-elevated flows even without admin rights on HKLM.
DLL hijacking concept	Abusing DLL search order so a process loads a malicious DLL instead of a legit one.	Watch for critical Windows processes loading unsigned DLLs alongside signed Microsoft DLLs (mixed signed/unsigned pattern).
Signed vs unsigned module loading	Native Windows DLLs are usually signed; malicious DLLs often are not.	Use Sysmon Image Load events to detect signed=true & signed=false DLLs under the same process, especially auto-elevated ones.
UACME-derived DLL names	UACME documents specific DLLs commonly abused (e.g., DismCore.dll , omctl32.dll , osksupport.dll , etc.).	Treat loading of these DLL names (especially unsigned, in odd paths) as strong evidence of UAC bypass attempts.
Trusted path collision (e.g., C:\Windows)	Creating fake system-like directories with trailing spaces (C:\Windows , C:\Program Files) to trick path parsing.	Any EXE/DLL creation in these visually similar but rogue directories is highly suspicious and usually tied to UAC bypass.
GetLongPathNameW normalization issue	AppInfo / Windows APIs can normalize paths and treat rogue dirs as real system paths.	Understand that path normalization bugs allow writing "inside" what Windows thinks is a trusted directory.
<u>Sysmon Event ID 1</u> – Process Creation	Logs process creation with parent/child info and integrity level.	Use it to detect Medium → High transitions, auto-approved parents spawning odd children, and elevated COM activations.
<u>Sysmon Event ID 7</u> – Image Loaded	Logs DLL loading (ImageLoaded , Signed , Company , etc.).	Use it to find unsigned DLLs in key processes, DLL hijacks, and known UACME-related DLL names.
<u>Sysmon Event ID 11</u> – FileCreate	Logs file creation (file_path) for executables and DLLs.	Monitor for EXE/DLL creation in rogue system-like directories (C:\Windows , C:\Program Files variants).
<u>Sysmon Event IDs 12/13/14</u> – Registry events	Log registry create/modify/delete for keys and values.	Use them to detect writes to critical UAC-related keys (e.g. ms-settings\Shell\Open\command , DelegateExecute).
<u>LUA-ConsentUI ETW provider</u>	Microsoft-Windows-LUA-ConsentUI/Diagnostic logs elevation attempts.	Correlate ETW events with Sysmon; ask whether elevation attempts align with normal prompts or suggest bypass/failure.
LUA-ConsentUI ETW EventID <u>16001</u> (executables)	Logs executable elevation attempts (PerfTrackId + EXE name).	Identify which binaries are trying to elevate; suspicious if unusual tools frequently appear or no normal consent is shown.
LUA-ConsentUI ETW EventID <u>16005</u> (COM classes)	Logs elevated COM class activations (PerfTrackId + COM friendly name).	Map COM friendly names to CLSIDs; spot unusual or repeated elevated COM activations as potential bypass avenues.

Area	What Students Must Understand	How to Reason About It (Detection Mindset)
Defender mindset	Focus on patterns of abuse , not specific PoCs or command lines.	Always ask: “Which trusted path is being abused (EXE/COM/registry/DLL/path), and what is the minimal, generic rule to catch it?”

Access Token Attacks

Area	What Students Must Understand	How to Reason About It (Attacker / Analyst Mindset)
Access tokens & security context	An access token defines who a process/thread is (user SID, groups, privileges) and thus what it can access.	Ask: “Which token is this code running under now, and what changes after impersonation?”
Process vs thread tokens	A process has a primary token; a thread can have its own impersonation token overriding the process token.	Remember: impersonation is usually per-thread ; one thread can act as SYSTEM while others in the same process stay as the original user.
<u>ImpersonateLoggedOnUser</u> high-level behavior	This API takes a token handle and makes the current thread impersonate that security context.	View it as: “Temporarily swap this thread’s identity with the token I give you”; if the token is SYSTEM, you now “become” SYSTEM.
Privilege requirement (SeDebugPrivilege)	To open sensitive processes like winlogon.exe and their tokens, the code first enables SeDebugPrivilege on its own token.	Realize that token attacks often start by enabling debug privileges to access high-privilege process tokens.
Targeting winlogon.exe	winlogon.exe runs as SYSTEM and holds a powerful primary token with many enabled privileges.	Reason: “If I can get a handle to winlogon’s token, I can impersonate SYSTEM and gain its full privileges.”
Using <u>OpenProcessToken</u>	OpenProcessToken on the winlogon handle returns a handle to its primary SYSTEM token .	This token handle is the critical asset : from here, duplication/impersonation gives you SYSTEM on your own thread.
Internal workings of <u>ImpersonateLoggedOnUser</u>	Internally calls NtQueryInformationToken , may NtDuplicateToken (to create an impersonation token), then NtSetInformationThread .	Understand that the API is a wrapper around lower-level Nt* calls: querying token type, duplicating if needed, then assigning to a thread.
Primary vs impersonation token types	Tokens can be primary (for processes) or impersonation (for threads).	ImpersonateLoggedOnUser: primary token → (optionally) duplicated into an impersonation token, then attached to the current thread.
<u>OpenThreadToken</u> after impersonation	Once impersonation is set, OpenThreadToken on the current thread returns the impersonation token handle .	This confirms: “My thread is now carrying an impersonation token; I can inspect it or locate its kernel address.”
Kernel “view” of process token	In kernel, the process token is referenced by an address (e.g., Token ffff... in !process).	Map user-mode handle to kernel token object ; helps understand that multiple handles can point to the same underlying token.

Area	What Students Must Understand	How to Reason About It (Attacker / Analyst Mindset)
Using <code>!process</code> and <code>!token</code> (WinDbg)	<code>!process</code> shows the primary token address; <code>!token</code> inspects token details (user SID, groups, privileges, type).	As an analyst: confirm the true identity and privilege set of the tokens being used, and whether they are primary or impersonation.
<code>_ETHREAD</code> and <code>ClientSecurity</code>	Each thread has a <code>_PS_CLIENT_SECURITY_CONTEXT</code> (<code>ClientSecurity</code>) storing impersonation info (token pointer, level, flags).	Understand that impersonation is encoded at the thread level , and can be inspected through <code>_ETHREAD</code> → <code>ClientSecurity</code> .
Comparing primary vs impersonation tokens	Both tokens may share the same SID , privileges, and Authentication ID; main difference is TokenType (Primary vs Impersonation).	This shows that impersonation tokens can be logically equivalent to the original (SYSTEM), just attached differently (process vs thread).
Authentication ID (<code>AuthenticationId</code>)	<code>AuthenticationId</code> ties the token to a logon session ; same ID implies same originating login.	Comparing Auth IDs helps verify that impersonated and primary tokens originate from the same SYSTEM logon session.
Modified ID (<code>ModifiedId</code>)	<code>ModifiedId</code> changes when a token is modified/duplicated; equal IDs mean <u>closely related tokens</u> .	Use it to reason about token cloning chains : duplicated tokens often keep the same Modified ID value.
Key privileges visible in the SYSTEM token	SYSTEM tokens commonly have strong privileges (e.g., <code>SeDebugPrivilege</code> , <code>SeImpersonatePrivilege</code> , <code>SeLoadDriverPrivilege</code> , etc.).	Understand that impersonating this token gives full capability to perform further privilege abuse across the system.
Defender perspective	Impersonation can be invisible at process level; only the thread may appear as SYSTEM.	As a defender, consider monitoring for token operations , access to SYSTEM processes/tokens, and <u>sudden</u> access to restricted resources.

Service Accounts Abuse

Area	What Students Must Understand	How to Reason About It (Detection / Attack Mindset)
Core telemetry sources	Detection relies mainly on Sysmon (<code>EventCode=1,3,10,17,18</code>) and optionally Zeek DCE/RPC , Security logs (RPC auditing), and ETW .	Map each detection concept to which log + Event ID is needed (process, network, process access, pipes, RPC).

SYSTEM Process Creation by Service Accounts

Area	What Students Must Understand	How to Reason About It
SYSTEM process with non-SYSTEM parent	SYSTEM processes normally have SYSTEM parents; in these attacks a service account (e.g. LOCAL SERVICE) spawns a SYSTEM process.	Use Sysmon Process Create (EventCode=1) where user=SYSTEM AND ParentUser != "NT AUTHORITY\\SYSTEM" to find suspicious escalations.
Service account → SYSTEM chain	Tools like SweetPotato run as a service account and end up creating a SYSTEM cmd.exe (or similar).	When you see LOCAL SERVICE or other service identities as parent of SYSTEM processes, treat it as a strong indicator of service-account escalation.
Why SweetPotato is used	SweetPotato bundles multiple techniques (PrintSpoofer, EfsRpc, etc.) to escalate from service accounts to SYSTEM.	Think of SweetPotato as a lab generator of “service-account → SYSTEM” events to validate your queries and reasoning.

JuicyPotato & Anonymous Logon Indicators

Area	What Students Must Understand	How to Reason About It
NTLM relay flow in JuicyPotato	JuicyPotato receives an NTLM auth from SYSTEM, uses AcceptSecurityContext to get a local SYSTEM token , and also relays it to RPCSS .	Understand that the same NTLM exchange is used both to impersonate SYSTEM locally and to talk to RPCSS on port 135.
Traffic to RPCSS on port 135	In the PCAP, JuicyPotato relays NTLM AUTH3 messages to RPCSS at localhost:135 .	Treat unusual local NTLM traffic to 135/RPCSS as a potential sign of JuicyPotato-type activity, especially on vulnerable OS versions.
Anonymous logon artifacts	Due to how the relay works, the system may record anonymous logons from localhost associated with these operations.	If you see anonymous logons combined with weird NTLM/RPC activity on 135, correlate them with process/network logs for potential Potato exploits.
OS version dependency	JuicyPotato works reliably only on pre-1809 / pre-Server 2019 systems.	Always factor in OS version: some techniques are blocked on modern builds, so detections there are more likely noise/experiments.

Named Pipe Abuse (RoguePotato / PrintSpoofer / PetitPotato)

Area	What Students Must Understand	How to Reason About It
Role of named pipes in these attacks	Modern Potato variants abuse named pipes to trick SYSTEM (or services) into authenticating and then impersonate that token.	View named pipes as local equivalents of network listeners used to capture or relay privileged authentication.
Sysmon pipe events (EventCode 17 & 18)	EventCode=17 = CreatePipe (server side), EventCode=18 = ConnectPipe (client side).	Use these to reconstruct who hosted the pipe and who connected , including user and process image.
High-value pipe names	Pipes like \\pipe\\srvsvc , \\pipe\\spoolss , \\pipe\\epmapper are tied to EfsRpc/Spooler/RPC endpoint mapper abuse.	Treat non-SYSTEM processes creating or using these pipes as high-signal events , given their role in coercion attacks.

Area	What Students Must Understand	How to Reason About It
Correlating server vs client users	Detect pipes where non-SYSTEM creates (EventCode=17) and SYSTEM connects (EventCode=18) for the same PipeName .	Pattern: “ <i>Low/medium account creates pipe, SYSTEM connects → likely token impersonation opportunity.</i> ”
SweetPotato as a generator	Running SweetPotato with -e PrintSpoofer creates a pipe, causes SYSTEM to connect, and spawns a SYSTEM shell.	Use this to validate the idea that server = service account / client = SYSTEM mapping appears in your aggregated Sysmon pipe events.

RoguePotato – External Resolver & ResolveOxid

Area	What Students Must Understand	How to Reason About It
Rogue OXID resolver concept	RoguePotato stands up a fake OXID resolver and a fake pipe endpoint, then tricks COM into using them for activation.	Understand that manipulating COM's ResolveOxid2 path lets the attacker redirect DCOM traffic to a controlled endpoint.
External DCE/RPC visibility	With Zeek DCE/RPC, you can see IObjectExporter operations (like ResolveOxid2) going to potentially external IPs/ports.	Query Zeek logs for unusual IObjectExporter / ResolveOxid traffic, especially to external or lab hosts, not typical DC/COM servers.
Local vs external focus	To reduce noise, local-only RPC may be excluded; focus on external ResolveOxid calls indicating hijacked activation paths.	Think: “ <i>Why is this host resolving OXIDs via an address that looks like a rogue resolver or a non-standard RPC endpoint?</i> ”
Service account execution context	RoguePotato is executed as a service account (e.g., LOCAL SERVICE) and ends up with a SYSTEM token.	Tie DCE/RPC telemetry to who is initiating the COM calls: service account + weird resolver endpoint = strong suspect.

Monitoring **CoGetInstanceFromIStorage** & COM Activation

Area	What Students Must Understand	How to Reason About It
CoGetInstanceFromIStorage in combase.dll	This API activates COM objects from a storage object and is central to RoguePotato-style COM abuse.	Treat unexpected or frequent use of CoGetInstanceFromIStorage from low-priv/service contexts as suspicious COM activation behavior.
Internal activator path	CoGetInstanceFromIStorage calls CComActivator::DoGetInstanceFromStorage after some TLS/flags checks.	Recognize there is a single internal path many external APIs converge on, making it a prime hook/monitoring target.

Area	What Students Must Understand	How to Reason About It
IStandardActivator::StandardGetInstanceFromIStorage	Another exported path (StandardGetInstanceFromIStorage) also routes into DoGetInstanceFromStorage .	Don't only monitor the top-level API: cover both external entry points so attackers can't trivially switch function names to evade monitoring.

API monitoring strategy

To get good coverage, monitor execution of both **CoGetInstanceFromIStorage** and **StandardGetInstanceFromIStorage**.

When designing hooks/ETW-based monitoring, think in terms of **internal implementation functions** rather than just one public API name.

RPC Abuse to Trigger SYSTEM Authentication

Area	What Students Must Understand	How to Reason About It
Targeted RPC interfaces (EfsRpc / Print Spooler)	Functions like EfsRpcOpenFileRaw , EfsRpcEncryptFileSrv , EfsRpcQueryUsersOnFile , RpcRemoteFindFirstPrinterChangeNotificationEx , etc.	These are common “ coercion ” APIs: forcing SYSTEM (or a DC) to authenticate somewhere (e.g., PetitPotam/EfsPotato chains).
Local / LPC limitation	Many of these RPC calls are over LPC/local RPC , so classic network logs may miss them.	Accept that network-only monitoring is insufficient; you need API-level or RPC endpoint auditing for full visibility.
RPC endpoint auditing via netsh rpc filter	You can configure filters for specific RPC interface UUIDs (e.g., EfsRpc UUID) and log access into the Security log.	Use these filters to focus exactly on high-risk interfaces instead of logging all RPC traffic (which is too noisy).
Security log visibility	Once auditing is enabled, Security events show the subject account and which RPC interface was accessed.	Correlate “which user called high-risk RPC” with process telemetry to determine if a service/low user is driving SYSTEM authentication.
Microsoft-Windows-RPC ETW	ETW provider for RPC can expose detailed RPC calls but with very high volume .	Use ETW selectively for lab or targeted hunts; apply filters by interface UUID / op number to keep data manageable.

RPCSS Process Access (RoguePotato)

Area	What Students Must Understand	How to Reason About It
Role of RPCSS	RPCSS (under svchost.exe -k RPCSS -p) is central to COM/DCOM; stealing its token is highly valuable.	RoguePotato impersonates NETWORK SERVICE and then uses access to RPCSS to obtain a SYSTEM token.

Area	What Students Must Understand	How to Reason About It
Sysmon EventCode 10 – Process Access	Process Access events show one process opening another, with user context for both source and target.	Monitor for non-SYSTEM processes (often as NETWORK SERVICE) accessing RPCSS as an indicator of token theft attempts.
Combining EventCode 10 & 1	Detection logic joins: (1) EventCode 10 where TargetUser="NT AUTHORITY\NETWORK SERVICE" and source is not SYSTEM, plus (2) EventCode 1 for RPCSS.	Aggregate by synthetic target_process_id and see multiple event types (access + create) pointing at the same RPCSS instance.
Distinct event count (dc > 1)	Counting distinct EventCodes per target lets you focus on processes involved in multiple suspicious actions , not just a single benign event.	Think in terms of multi-stage behavior : access + existence of RPCSS together gives context that flags RoguePotato.exe as the attacker process.

Rogue WinRM Listener

Area	What Students Must Understand	How to Reason About It
Rogue WinRM pattern	RogueWinRM.exe sets up a fake WinRM listener on TCP 5985 , then coerces BITS / service accounts to authenticate to it and gains SYSTEM.	Any non-standard process listening on 5985 and receiving local connections should be treated as a rogue WinRM endpoint .
Sysmon EventCode 3 – Network Connect	Network events show connections with src_ip, dest_ip, dest_port, Image, Initiated .	Filter for EventCode=3, dest_port=5985, Initiated=false to identify listeners instead of clients.
Aggregation by process_id	Group by process_id and collect IPs, ports, Images, etc., to see which process is acting as a WinRM server.	If a public-path binary (e.g., C:\Users\Public\Executables\RogueWinRM.exe) consistently shows up as receiving on 5985, raise an alert.
Handling <unknown process> image	Sysmon sometimes logs <unknown process> in network events; correlating with process creation (EventCode=1) recovers the true Image.	Always join network events with process create events by PID to get reliable process names and command lines.
WFP as alternative source	Windows Filtering Platform logs can also show inbound 5985 listeners and connections.	Use WFP where Sysmon is unavailable, or as a cross-check in environments with strict network logging policies.
BITS & WinRM preconditions	For RogueWinRM to work, default WinRM service must be stopped (to free 5985) and BITS restarted to trigger auth to the rogue listener.	On a compromised host, look for WinRM stop + BITS activity + rogue listener on 5985 as a complete chain of suspicious behavior.

Defender Mindset

Area	What Students Must Understand	How to Reason About It
Cross-surface correlation	Service account → SYSTEM escalation uses a mix of process, pipe, RPC, and network techniques.	Don't silo detections: combine SYSTEM process creation, named pipe patterns, RPCSS access, and WinRM listeners for high-confidence alerts.
Behavioral vs tool-specific detection	Good rules detect patterns (e.g., non-SYSTEM → SYSTEM, rogue pipes, external ResolveOxid, rogue 5985 listeners), not just known tool names.	If an attacker rewrites SweetPotato/RoguePotato/RogueWinRM, your detections should still trigger because the underlying behavior hasn't changed.
Lab tools as telemetry generators	SweetPotato, RoguePotato, RogueWinRM are used in the lab to generate realistic telemetry for each pattern.	Use them to practice mapping observed logs back to the underlying technique: "what log pattern proves this privilege escalation happened?"

Commonly-abused **Privileges** during Token-related Attacks

Privilege Name	Symbolic Constant	How It Impacts Access Token Attacks
Impersonate a client after authentication	<u>SeImpersonatePrivilege</u>	Allows a process to adopt the security context of an authenticated client (e.g., over named pipes/COM). In Potato-style attacks, once SYSTEM authenticates to the attacker-controlled service, the attacker impersonates the SYSTEM token and escalates to full system privileges.
Replace a process level token	<u>SeAssignPrimaryTokenPrivilege</u>	Lets a process assign a different primary token to itself or another process. After stealing or duplicating a privileged token, an attacker can start or retokenize a process with that token and effectively run as SYSTEM or another high-privileged account.
Create a token object	<u>SeCreateTokenPrivilege</u>	Permits the creation of new access tokens in user mode. An attacker with this right can craft tokens containing arbitrary SIDs and privileges (including SYSTEM or domain admin).
Debug programs	<u>SeDebugPrivilege</u>	Authorizes a process to open and fully control any other process, including SYSTEM-owned ones. This enables token theft and duplication from privileged processes, as well as code injection that manipulates tokens to spawn processes under elevated security contexts.
Load and unload device drivers	<u>SeLoadDriverPrivilege</u>	Allows loading and unloading kernel-mode drivers, including malicious/vulnerable drivers that may allow an attacker to gain kernel-level write/read/execute permissions and thus read, modify, or forge access tokens (nt!_TOKEN) in kernel's memory.